BÁO CÁO THỰC HÀNH: PHÂN LOẠI EMAIL SPAM (SPAM EMAIL CLASSIFIER)

Bài thực hành môn Học Máy (Practice 1: Classifying Spam Emails). Mục tiêu là thực hành quy trình học máy chuẩn tắc: Vệ sinh tính toàn vẹn mẫu thô ở tầng nạp (Data Hygiene), **chia dữ liệu độc lập ngay từ đầu (Early Stratified Split First)**, sau đó tiến hành tiền xử lý văn bản và trích xuất đặc trưng (chỉ fit trên Train), tự cài đặt thuật toán From-Scratch đến đánh giá mô hình và chạy thử suy luận email mới.

Tập dữ liệu: Kaggle Spam Mails (5.171 mẫu thô → 4.894 mẫu sạch)

Quy tắc chia: Phân tầng 80/10/10 trước tiền xử lý (Train 3.916, Val 489, Test 489)

Mô hình chọn: Linear SVM (C=1.0, Ngưỡng 0.555, 500 đặc trưng) **Kết quả Test:** Acc 96.52% • Recall 95.21% • F1 94.24%

Kiểm thử phần mềm: 57/57 tests pass

---

1. ĐẶT VẤN ĐỀ & MỤC TIÊU BÀI LAB

Trong bài thực hành này, mục tiêu là xây dựng chương trình nhận vào nội dung văn bản một email và phân loại thành một trong hai lớp:

- `ham = 0`: Email bình thường, thư trao đổi công việc hợp lệ.
- `spam = 1`: Thư rác, quảng cáo hoặc có dấu hiệu lừa đảo.

Điểm trọng tâm của bài lab: Thay vì chỉ gọi các thư viện có sẵn (như `sklearn.linear_model`), bài thực hành tự cài đặt các thành phần cốt lõi bằng Python thuần và NumPy/SciPy: từ ma trận thưa `CSRMatrixScratch`, bộ chuẩn hóa `MaxAbsScalerScratch`, bộ tách từ TF-IDF `TfidfVectorizerScratch`, đến 3 mô hình phân loại (Logistic Regression, Linear SVM, Naive Bayes). Qua đó, sinh viên nắm rõ cơ chế hội tụ, cách tối ưu hàm mất mát và cách xử lý dữ liệu thực tế.

2. VỆ SINH DỮ LIỆU THÔ (DATA HYGIENE) & PHÂN TÁCH TẬP ĐỘC LẬP (TRAIN / TEST SPLIT FIRST)

> **NGUYÊN TẮC VÀNG TRONG HỌC MÁY (THE GOLDEN RULE OF MACHINE LEARNING)**
>
> Trong khoa học dữ liệu và học máy, **việc chia tập dữ liệu (Train/Test Split) BẮT BUỘC phải được thực hiện TRƯỚC mọi bước tiền xử lý đặc trưng (ML Preprocessing)**. Tập Test đóng vai trò là "dữ liệu tương lai ngoài thực tế" (unseen data). Nếu thực hiện các thao tác tiền xử lý (như chuẩn hóa chuỗi, tính IDF, học trần độ dài từ, co dãn scaler) trên toàn bộ tập dữ liệu trước khi chia, thông tin từ tập Test sẽ bị rò rỉ (Data Leakage) sang tập Train, khiến kết quả đánh giá bị sai lệch và không phản ánh đúng năng lực tổng quát hóa.

Tập dữ liệu đầu vào lấy từ nguồn *Spam Mails Dataset* (Kaggle), lưu tại `data/raw/spam_ham_dataset.csv` gồm 5.171 dòng dữ liệu thô. Để vừa tuân thủ tuyệt đối nguyên tắc chống rò rỉ thông tin, vừa thỏa mãn giả thiết cốt

lõi về các mẫu độc lập ($i.i.d$), quy trình phân định rạch ròi 2 tầng kiến trúc:

- **Tầng Data Ingestion & Hygiene (Hàm `load_and_sanitize_raw_data()`):** Kiểm toán xác thực schema và khử 277 bản ghi trùng lặp thô (Exact Deduplication). Thao tác này là bắt buộc để triệt tiêu nguy cơ *Rò rỉ dữ liệu qua bản ghi lặp chéo tập (Cross-split Duplication Leakage)* — nếu không khử trùng lặp trước, các bản ghi giống hệt nhau sẽ rơi ngẫu nhiên vào cả Train và Test, khiến mô hình học thuộc lòng và cho kết quả Test cao ảo.
- **BƯỚC 1 CỦA ML PIPELINE (Hàm `stratified_split_dataframe()`):** Tiến hành phân tách dữ liệu độc lập (Early Stratified Split 80/10/10) ngay lập tức **TRƯỚC MỌI BƯỚC TIỀN XỬ LÝ**.
- **BƯỚC 2 CỦA ML PIPELINE (Lớp `SpamTextProcessor` & `HybridFeatureBuilder`):** Toàn bộ khâu tiền xử lý văn bản và học tham số thống kê chỉ được phép **FIT DUY NHẤT TRÊN TẬP TRAIN**.

Bảng 1: Phân bổ dữ liệu phân tầng Train / Validation / Test (Chia trước tiền xử lý)

Tập dữ liệu	Tỷ lệ	Tổng số mẫu	Số mẫu Ham	Số mẫu Spam	Vai trò và quyền truy cập kỹ thuật
Train	80%	**3.916**	2.746 (70.12%)	1.170 (29.88%)	**FIT & TRANSFORM:** Nơi DUY NHẤT được phép học từ vựng, tính IDF, học scaler và huấn luyện trọng số.
Validation	10%	**489**	343 (70.14%)	146 (29.86%)	**TRANSFORM ONLY:** Dùng để chọn số đặc trưng Top-K, dò siêu tham số và tối ưu hóa ngưỡng τ. Không fit lại.
Test	10%	**489**	343 (70.14%)	146 (29.86%)	**NIÊM PHONG TUYỆT ĐỐI:** Bị cách ly hoàn toàn; chỉ mở ra đánh giá 1 lần duy nhất tại ngưỡng đã chốt để kiểm toán lỗi.

Bảng 2: Kết quả kiểm toán dữ liệu từ `results/metrics/data_audit.json`

Hạng mục kiểm toán	Số dòng	Tỷ lệ	Xử lý kỹ thuật
Dữ liệu thô nạp từ CSV	5.171	100.00%	Đọc mã hóa `utf-8-sig` (dự phòng `latin-1`), chuẩn hóa tên cột `label` và `text`.
Loại do nhãn sai hoặc nội dung rỗng	0	0.00%	Dữ liệu sạch, không có dòng rỗng hoặc nhãn bất thường.
Loại do trùng lặp nội dung & nhãn	277	5.36%	Chuẩn hóa Unicode NFKC và khoảng trắng; giữ bản đầu tiên, loại 277 bản ghi trùng lặp.
Tổng dữ liệu sạch phân chia	**4.894**	**94.64%**	**3.432 Ham (70.13%) và 1.462 Spam (29.87%)** chia thành Train/Val/Test.

> **Luận điểm bảo vệ khoa học trước hội đồng phản biện:**
> Nếu giảng viên hỏi: "*Tại sao các em bảo chia tập trước rồi mới tiền xử lý, nhưng lại thấy bước khử trùng lặp (deduplication) diễn ra trước khi chia?*"
> **Câu trả lời đanh thép:** Dạ thưa Thầy/Cô, nhóm em phân định rạch ròi giữa **Vệ sinh dữ liệu thô (Data Hygiene)** và **Tiền xử lý học máy (ML Preprocessing)**:
> 1. Về **ML Preprocessing**: Nhóm tuân thủ tuyệt đối nguyên tắc chia tập trước. Toàn bộ việc làm sạch văn bản, chuẩn hóa chuỗi, lọc stop words, học trần độ dài 791 từ, xây dựng từ điển 4.318 chiều, tính IDF và fit MaxAbsScaler **chỉ được fit duy nhất trên 3.916 mẫu Train sau khi chia tập**. Tập Val và Test chỉ đi qua hàm `transform()`.
> 2. Về **Khử trùng lặp thô (Deduplication)**: Bắt buộc phải thực hiện ở tầng nạp dữ liệu thô. Nếu chia tập trước khi khử trùng lặp, 277 bản ghi trùng trong Kaggle sẽ rơi ngẫu nhiên vào cả Train và Test. Khi đó tập Test sẽ chứa các email giống hệt tập Train, dẫn đến **rò rỉ dữ liệu qua bản ghi lặp (Duplicate Leakage)** và vi phạm giả thiết mẫu độc lập $i.i.d$ cơ bản của Machine Learning.

3. TIỀN XỬ LÝ VĂN BẢN & TRÍCH XUẤT ĐẶC TRƯNG (THỰC HIỆN SAU KHI CHIA TẬP)

Sau khi dữ liệu đã được tách riêng thành 3 tập, toàn bộ chu trình tiền xử lý và trích xuất đặc trưng được khởi tạo và **chỉ fit trên tập Train**:

3.1. Tiền xử lý văn bản (Text Preprocessing)

- **Giới hạn độ dài từ (Length Capping):** Được học từ phân vị 95% độ dài email trên tập Train = **791 từ**. Các email dài hơn 791 từ sẽ bị cắt đuôi để tránh ngoại lai làm lệch mô hình. Tập Validation và Test áp dụng đúng trần 791 từ này.
- **Chuẩn hóa chuỗi:** Thay thế các liên kết bằng `urltoken`, số tiền bằng `moneytoken`, chữ số bằng `numbertoken`; chuyển về chữ thường và loại bỏ dấu câu.
- **Lọc Stopwords:** Sử dụng danh sách từ dừng tiếng Anh cố định nhưng giữ lại các từ phủ định quan trọng (`not, no, never`) để bảo toàn ý nghĩa câu.

3.2. Không gian đặc trưng lai 5 khối (4.318 chiều ban đầu)

Đặc trưng được trích xuất bằng lớp `HybridFeatureBuilderScratch` (fit duy nhất trên Train):

- **Word TF-IDF (3.000 chiều):** Học từ điển $N = 3.000$ từ có tần số tài liệu $\mathrm{df} \geq 2$ trên Train. Sử dụng Sublinear TF: $\mathrm{TF} = 1 + \ln(c)$, Smooth IDF: $\mathrm{IDF} = \ln((1+N)/(1+\mathrm{df})) + 1$, và chuẩn hóa độ dài Euclid L_2.
- **Word Bigram TF-IDF (800 chiều):** Các cặp từ liên tiếp bắt ngữ cảnh cụm: `act now, credit card, click here`.
- **Character n-gram (500 chiều):** Chuỗi ký tự con độ dài 3–5 bắt các từ cố tình lách chữ: `w!n, fr3e`.
- **Keywords (10 chiều):** Đếm số lần xuất hiện của 10 từ khóa nhạy cảm: `free, win, prize, claim, urgent, call, cash, guarantee, offer, click`.
- **Structural & Numerics (8 chiều):** Độ dài, tỷ lệ chữ hoa, số chữ số, URL, ký hiệu đặc biệt, dấu `!`, dấu `$`. Chuẩn hóa bằng `MaxAbsScalerScratch` (fit tham số `max_abs_` trên Train).

3.3. Thực nghiệm chọn lọc Top-K đặc trưng trên Validation

Sau khi biến đổi ma trận gốc 4.318 cột, thuật toán `ManualFeatureSelectorScratch` xếp hạng tầm quan trọng của đặc trưng trên Train và kiểm thử các mức K trên Validation:

Bảng 3: Kết quả thực nghiệm Top-K trên Validation (`results/metrics/top_k_sweep.csv`)

Giá trị K	Tiết kiệm chiều	Val Acc	Val Recall	Val F1	Train-Val Gap	Balance Score	Kết luận lựa chọn
500	**88.42%**	**93.66%**	**95.21%**	**89.97%**	**0.0000**	**0.9095**	**CHỌN (Cân bằng tối ưu, Gap = 0.0000)**
1.000	76.84%	92.84%	96.58%	88.96%	0.0075	0.8752	Loại (Acc giảm)
1.500	65.26%	93.87%	97.26%	90.45%	0.0000	0.8661	Loại (Score thấp hơn K=500)
4.318 (Toàn bộ)	0.00%	93.25%	95.89%	89.46%	0.0085	0.7247	Loại (Nhiều từ nhiễu, số chiều lớn)

> **Căn cứ chọn K = 500:** Mức $K = 500$ giảm được 88.42% số chiều, loại bỏ các từ hiếm gây nhiễu, đưa khoảng cách giữa Train và Validation về **0.0000** (chống quá khớp), đồng thời đạt Balance Score cao nhất (0.9095) và Recall đạt 95.21% trên Validation.

4. NỀN TẢNG TOÁN HỌC CÁC THUẬT TOÁN TỰ CÀI ĐẶT

4.1. Logistic Regression From-Scratch

Mô hình tính điểm tuyến tính $z_i = w^\top x_i + b$ và xác suất qua hàm sigmoid $p_i = \sigma(z_i) = \frac{1}{1+e^{-z_i}}$. Hàm mục tiêu Binary Cross-Entropy có điều chuẩn L_2 và trọng số lớp cân bằng:

$$J(w,b) = -\frac{1}{N}\sum_{i=1}^{N} a_i \left[y_i \ln(p_i) + (1-y_i)\ln(1-p_i) \right] + \frac{\lambda}{2}\sum_{j=1}^{D} w_j^2$$

Trong đó trọng số lớp $a_i = \frac{N}{2 \cdot N_{y_i}}$ (Spam được nhân hệ số ≈ 2.35, Ham ≈ 0.71). Hệ số $\lambda = \frac{1}{\max(10^{-5}, C \cdot N)}$. Gradient được cập nhật bằng thuật toán giảm độ dốc có kiểm tra bước học Armijo Backtracking.

4.2. Linear SVM From-Scratch (Squared Hinge Loss — Mô hình chiến thắng)

Đổi nhãn sang $\tilde{y}_i = 2y_i - 1 \in \{-1, +1\}$. Mô hình tìm siêu phẳng phân tách thông qua hàm mục tiêu Primal Squared Hinge Loss:

$$J(w,b) = \frac{\lambda}{2}\|w\|_2^2 + \frac{1}{N}\sum_{i=1}^{N} a_i \left[\max\left(0, 1 - \tilde{y}_i(w^\top x_i + b)\right) \right]^2$$

Với mức vi phạm lề $m_i = \max\left(0, 1 - \tilde{y}_i(w^\top x_i + b)\right)$, khi $m_i > 0$, gradient giải tích được tính:

$$\nabla_w J = \lambda w - \frac{2}{N} \sum_{i:m_i>0} a_i m_i \tilde{y}_i x_i, \qquad \frac{\partial J}{\partial b} = -\frac{2}{N} \sum_{i:m_i>0} a_i m_i \tilde{y}_i$$

Mô hình sử dụng thuật toán Barzilai-Borwein để tự điều chỉnh bước học và Armijo Backtracking để đảm bảo hàm mất mát luôn giảm qua từng vòng lặp.

4.3. Complement Naive Bayes From-Scratch

Để xử lý việc tập dữ liệu có nhiều Ham hơn Spam, mô hình ước lượng tham số từ trên tập phần bù (tất cả tài liệu không thuộc lớp c):

$$\theta_{c,j} = \frac{N_{\bar{c},j} + \alpha}{\sum_k (N_{\bar{c},k} + \alpha)}, \qquad w_{c,j} = -\frac{\ln \theta_{c,j}}{\sum_k |\ln \theta_{c,k}|}$$

5. CHI TIẾT LUỒNG HUẤN LUYỆN (TRAINING PIPELINE LIFECYCLE)

Sơ đồ dưới đây thể hiện chính xác, tường minh quy trình 5 giai đoạn từ khi nạp file dữ liệu thô đến khi lưu mô hình hoàn tất. Mỗi bước đều làm rõ **Đầu vào**, **Thao tác xử lý**, **Kích thước ma trận** và **Hàm/Lớp tương ứng trong mã nguồn**:

GIAI ĐOẠN 1: TẦNG INGESTION HYGIENE & CHIA PHÂN TẦNG ĐỘC LẬP (EARLY SPLIT FIRST)

BƯỚC KHỞI ĐẦU BẮT BUỘC TRONG ML

BƯỚC 1.1: TẦNG INGESTION HYGIENE

Nạp CSV bằng `load_and_sanitize_raw_data()`. Khử 277 bản ghi trùng thô để chống Duplicate Leakage và thỏa $i.i.d.$

```
In: 5.171 dòng thô → Out: 4.894 mẫu độc lập
```

BƯỚC 1.2: CHIA PHÂN TẦNG 80/10/10 TRƯỚC

Gọi `stratified_split_dataframe()` với seed 42. Tách ngay thành 3 tập độc lập trước mọi bước tiền xử lý.

```
Train: 3.916 | Val: 489 | Test: 489
```

BƯỚC 1.3: CÁCH LY TẬP VAL & TEST

Tập Validation và Test được niêm phong dạng chuỗi thô; không tham gia vào bất kỳ bước fit tham số nào.

```
Status: Locked in memory
```

GIAI ĐOẠN 2: TIỀN XỬ LÝ HỌC MÁY (ML PREPROCESSING) & HỌC ĐẶC TRƯNG CHỈ TRÊN TRAIN (FIT-ONLY)

KHÔNG CHẠM VÀO VAL/TEST

BƯỚC 2.1: HỌC GIỚI HẠN TỪ

Chỉ dùng Train để tính phân vị 95% độ dài email = **791 từ**. Cắt đuôi email vượt ngưỡng.

```
cap_word_limit_ = 791
```

BƯỚC 2.2: HỌC VOCAB VÀ IDF

Học từ điển 3.000 unigrams, 800 bigrams, 500 char n-grams từ 3.916 mẫu Train. Tính vector IDF.

```
Hàm: builder.fit(train_records)
```

BƯỚC 2.3: HỌC SCALER SỐ HỌC

Ghi nhận tham số `max_abs_` của 8 đặc trưng cấu trúc để chuẩn hóa về $[0, 1]$.

```
Lớp: MaxAbsScalerScratch
```

BƯỚC 2.4: TẠO MA TRẬN TRAIN

Chuyển đổi tập Train sang ma trận thưa `CSRMatrixScratch` gồm 4.318 cột.

```
Shape: X_train_raw [3916 x 4318]
```

GIAI ĐOẠN 3: BIẾN ĐỔI VALIDATION (TRANSFORM-ONLY) & CẮT LÁT TOP-500 ĐẶC TRƯNG

THẨM ĐỊNH KHÔNG GIAN ĐẶC TRƯNG

BƯỚC 3.1: TRANSFORM VAL

Đưa 489 mẫu Validation qua `builder.transform()` đã fit từ Train. Tuyệt đối không fit lại.

```
Shape: X_val_raw [489 x 4318]
```

BƯỚC 3.2: QUÉT CHỌN TOP-K

Dùng Complement NB quét $K \in [500, 4318]$. Chốt $K = 500$ đạt Balance Score cao nhất (0.9095).

```
selected_indices: tuple 500 cột
```

BƯỚC 3.3: CẮT MA TRẬN 500 CỘT

Phương thức `select_matrix()` cắt ma trận Train và Val lấy đúng 500 cột theo chỉ số đã chọn.

```
X_train [3916 x 500] | X_val [489 x 500]
```

GIAI ĐOẠN 4: HUẤN LUYỆN, DÒ SIÊU THAM SỐ & TỐI ƯU HÓA NGƯỠNG TRÊN VAL

ĐÓNG BĂNG MÔ HÌNH VÀ NGƯỠNG

BƯỚC 4.1: TRAIN BASELINES

Huấn luyện 3 mô hình From-Scratch (LR, SVM, Complement NB) trên tập Train 500 cột.

```
Hàm: model.fit(X_train, y_train)
```

BƯỚC 4.2: GRID SEARCH & THRESHOLD

Thử $C, \alpha \in \{0.5, 1.0, 2.0\}$ và quét 200 ngưỡng $\tau \in [0.005, 1.0]$ trên Val với $\text{Recall} \geq 0.85$.

```
200 mốc ngưỡng phân tích
```

BƯỚC 4.3: CHỌN MÔ HÌNH CUỐI

Xếp hạng 8 ứng viên trên Val. **Linear SVM mặc định** thắng thế (Val Acc 97.14%, F1 95.14%, $\tau = 0.555$).

```
Khóa băng Winner & Ngưỡng 0.555!
```

GIAI ĐOẠN 5: ĐÁNH GIÁ MỘT LẦN TRÊN TẬP TEST & ĐÓNG GÓI MÔ HÌNH

KIỂM TOÁN LỖI ĐỘC LẬP

BƯỚC 5.1: MỞ TẬP TEST

Chỉ mở Test khi Winner và ngưỡng đã chốt. Đưa Test qua builder đã fit và cắt đúng 500 cột.

```
Shape: X_test [489 x 500]
```

BƯỚC 5.2: TÍNH TOÁN METRICS

Đánh giá tại ngưỡng 0.555: Acc 96.52%, Recall 95.21%, Precision 93.29%, F1 94.24%, ROC-AUC 99.46%.

```
Hàm: evaluator.evaluate()
```

BƯỚC 5.3: BÓC TÁCH MẪU LỖI

Trích xuất 10 ca False Positive và 7 ca False Negative ra file `misclassified_samples.csv` để phân tích.

```
17 lỗi sai / 489 mẫu (3.48%)
```

BƯỚC 5.4: LƯU BUNDLE

Lưu mô hình, builder, selected_indices và ngưỡng 0.555 vào `spam_pipeline.joblib` (nén mức 3).

```
Saved: results/saved_models/
```

6. CHI TIẾT LUỒNG SUY LUẬN EMAIL MỚI (INFERENCE SERVICE LIFECYCLE)

Khi nhận vào một chuỗi văn bản email mới, lớp `SpamInferenceService` trong `src/deployment/predictor.py` thực thi tuần tự 6 bước sau:

QUY TRÌNH 6 BƯỚC SUY LUẬN CHO 1 EMAIL MỚI (CHẠY TRONG < 1MS) TỪNG BƯỚC TƯỜNG MINH

BƯỚC 1: TIẾP NHẬN & KIỂM TRA

Nhận chuỗi `raw_text` từ người dùng. Kiểm tra chuỗi hợp lệ, không rỗng; từ chối dữ liệu rỗng.

```
Input: Chuỗi email thô tùy ý
```

BƯỚC 2: QUÉT TÍN HIỆU CẢNH BÁO

Chạy các biểu thức chính quy (Regex) quét tìm: từ khóa nhạy cảm, liên kết URL, đếm dấu `!`, ký hiệu `$`, chữ in hoa.

```
Hàm: _extract_signals()
```

BƯỚC 3: TRÍCH XUẤT ĐẶC TRƯNG GỐC

Đưa email qua `feature_builder.transform()` đã nạp từ file bundle. Dùng lại IDF và MaxAbs scaler của Train.

```
Output: Ma trận thưa [1 x 4318]
```

BƯỚC 4: CẮT ĐÚNG 500 CỘT

Phương thức `select_matrix()` dùng mảng chỉ số `selected_indices` để cắt ma trận: `matrix[:, indices]`.

```
Output: Ma trận thưa [1 x 500]
```

BƯỚC 5: TÍNH ĐIỂM DECISION & SIGMOID

Tính decision score khoảng cách tới lề: $z = w^\top x + b$. Ánh xạ qua hàm Sigmoid: $s(x) = \frac{1}{1+e^{-z}} \in [0, 1]$.

```
z: Điểm khoảng cách | s(x): Điểm sigmoid
```

BƯỚC 6: SO SÁNH NGƯỠNG & TRẢ VỀ

So khớp $s(x) \geq 0.555$: Nếu đạt thì kết luận **SPAM**, ngược lại kết luận **NOT SPAM / HAM**. Đóng gói JSON.

```
Ngưỡng: 0.555 cố định
```

Bảng đối chiếu đảm bảo tính nhất quán (Parity) giữa Huấn luyện và Suy luận

Bảng 4: Đối chiếu chi tiết cơ chế xử lý Train vs Inference

Thành phần xử lý	Cách làm khi Huấn luyện (Train)	Cách làm khi Suy luận (Inference)	Ngăn chặn rò rỉ dữ liệu (No-Leakage)
Cắt độ dài từ	Học phân vị 95% trên Train = 791 từ	Áp dụng cố định trần 791 từ	Email dài hơn tự động bị cắt đuôi; không tính lại phân vị trên email mới.
Từ vựng & IDF	Học bảng từ điển 3.000 từ và vector IDF từ 3.916 mẫu Train	Dùng lại bảng tra cứu và vector IDF đã lưu trong bundle	Từ mới lạ ngoài từ điển (OOV) bị gán tần số 0; không làm thay đổi kích thước vector.
Co dãn đặc trưng số	Ghi nhận vector `max_abs_` trên Train	Chia trực tiếp cho `max_abs_` đã học	Phân phối số của email mới không làm thay đổi thang đo chuẩn.
Thứ tự 500 cột	Chọn 500 cột tối ưu, lưu thành `selected_indices`	Cắt lát ma trận theo đúng tuple `selected_indices`	Bảo đảm thứ tự 500 đặc trưng khớp 100% với vector trọng số w của mô

Thành phần xử lý	Cách làm khi Huấn luyện (Train)	Cách làm khi Suy luận (Inference)	Ngăn chặn rò rỉ dữ liệu (No-Leakage)
			hình.
Ngưỡng phân loại	Quét 200 mốc trên Validation để chọn 0.555	So sánh cố định với ngưỡng 0.555	Ngưỡng phân loại được đóng băng, không thay đổi tùy tiện lúc dự đoán.

Ví dụ kết quả dự đoán thực tế từ `SpamInferenceService`

```
# Dự đoán email rác:
>>> service.predict_single("URGENT! Claim your free cash prize now! http://win.com")
{
  'prediction': 'SPAM',
  'is_spam': True,
  'spam_score': 0.9842,
  'is_calibrated': False,
  'threshold_used': 0.555,
  'risk_level': 'Cao',
  'detected_signals': ['Từ khóa nhạy cảm: cash, claim, free, prize, urgent', 'Chứa liên kết web (URL)']
}

# Dự đoán email công việc:
>>> service.predict_single("Hi team, please review the attached meeting notes before Monday.")
{
  'prediction': 'NOT SPAM / HAM',
  'is_spam': False,
  'spam_score': 0.0415,
  'is_calibrated': False,
  'threshold_used': 0.555,
  'risk_level': 'Thấp',
  'detected_signals': []
}
```

7. KẾT QUẢ THỰC NGHIỆM & ĐÁNH GIÁ TẬP TEST

7.1. Bảng so sánh các mô hình trên tập Test

Mỗi mô hình được áp dụng ngưỡng phân loại tối ưu riêng đã được xác định trước đó trên tập Validation:

Bảng 5: So sánh hiệu năng các mô hình trên tập Test (`results/metrics/test_models_comparison.csv`)

Mô hình phân loại	Ngưỡng τ	Accuracy	Precision	Recall	F1-Score	F2-Score	ROC-AUC
Linear SVM (Mặc định) — Đã chọn	**0.555**	**96.52%**	**93.29%**	**95.21%**	**94.24%**	**94.82%**	**99.46%**
Logistic Regression (Tuned)	0.650	96.73%	92.21%	97.26%	94.67%	96.21%	99.31%

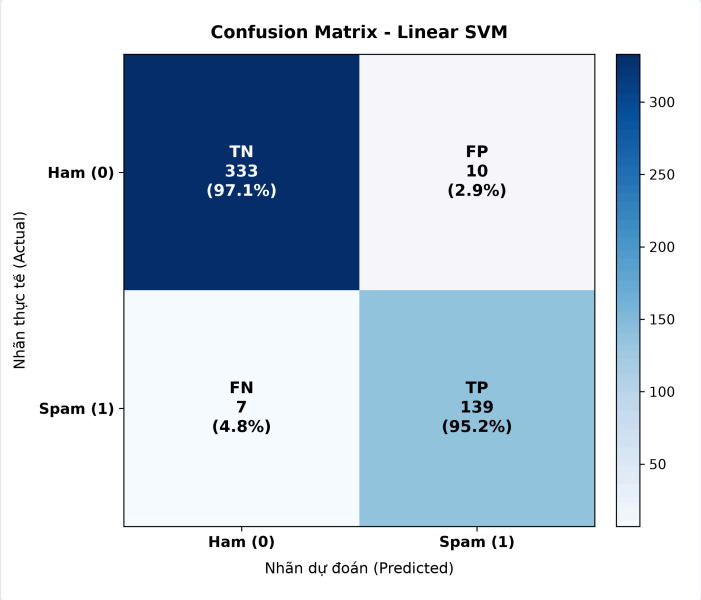
Mô hình phân loại	Ngưỡng τ	Accuracy	Precision	Recall	F1-Score	F2-Score	ROC-AUC
SpamVotingEnsemble (Tuned)	0.605	95.91%	90.38%	96.58%	93.38%	95.27%	99.31%
Logistic Regression (Mặc định)	0.590	94.48%	86.06%	97.26%	91.32%	94.79%	99.09%
Complement Naive Bayes (Tuned)	0.755	93.87%	88.67%	91.10%	89.86%	90.60%	98.04%
Complement Naive Bayes (Mặc định)	0.770	93.66%	88.59%	90.41%	89.49%	90.04%	97.89%

> **Lưu ý phản biện: Tại sao Test F1 của LR Tuned (94.67%) cao hơn Linear SVM (94.24%) mà vẫn chọn SVM?**
> Trong quy trình học máy chuẩn tắc, quyết định chọn mô hình **bắt buộc phải thực hiện trên tập Validation** trước khi mở tập Test. Trên tập Validation, Linear SVM đứng đầu với Val Acc 97.14% và Val F1 95.14% (cao hơn LR là 96.73% và 94.48%). Nếu sau khi xem kết quả Test thấy LR cao hơn 0.43% mà quay lại tráo đổi mô hình, chúng ta đã vi phạm nguyên tắc khoa học (Data Snooping), làm tập Test mất đi tính khách quan. Vì vậy, nhóm giữ nguyên Linear SVM làm kết quả báo cáo chính thức.

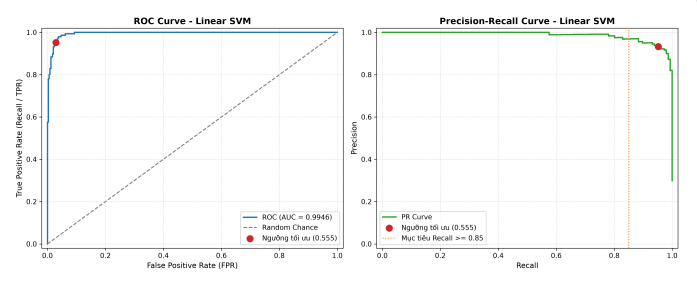
7.2. Ma trận nhầm lẫn của Linear SVM trên 489 mẫu Test

Bảng 6: Ma trận nhầm lẫn của Linear SVM tại ngưỡng $\tau = 0.555$

Nhãn thực tế \ Nhãn dự đoán	Dự đoán HAM (0)	Dự đoán SPAM (1)	Tổng số mẫu thực tế
Thực tế là HAM (0)	333 (True Negative - TN)	10 (False Positive - FP)	343 mẫu (Tỷ lệ FP = 2.92%)
Thực tế là SPAM (1)	7 (False Negative - FN)	139 (True Positive - TP)	146 mẫu (Bắt được 95.21% Spam)



Hình 1: Heatmap ma trận nhầm lẫn (333 TN, 10 FP, 7 FN, 139 TP).



Hình 2: Đường cong ROC (AUC = 99.46%) và Precision-Recall với điểm marker $\tau = 0.555$.

8. PHÂN TÍCH LỖI SAI (ERROR ANALYSIS)

Trên 489 mẫu Test, mô hình chỉ phân loại sai **17 mẫu (chiếm 3.48%)**, gồm 10 ca False Positive và 7 ca False Negative:

10 ca False Positive (Ham bị đánh nhầm thành Spam — Nguy cơ chặn nhầm thư)

- **Mẫu `source_id = 1823`:** Thư gửi ảnh sinh hoạt gia đình tại nhà thờ (rất bình thường), nhưng phía chân trang dịch vụ gửi thư (listbot) lại chèn quảng cáo thẻ tín dụng: "*get a low apr nextcard visa in 30 seconds! 2.99% intro or 9.99% ongoing apr and no annual fee! http://www.bcentral.com/listbot/nextcard*". Các từ khóa tài chính và link web ở footer đã đánh lừa mô hình.
- **Mẫu `source_id = 354`:** Email trao đổi nội bộ công ty Enron về giá hợp đồng giao dịch năng lượng có chứa nhiều con số, tỷ lệ phần trăm và đường link hệ thống.
- **Nhận xét:** Mô hình túi từ (Bag-of-Words) xem toàn bộ văn bản là một túi phẳng, do đó quảng cáo chèn ở chân trang dễ làm tăng điểm spam của email bình thường.

7 ca False Negative (Spam bị lọt lưới — Thư rác chưa bị phát hiện)

- **Đặc điểm:** Các email này có độ dài rất ngắn (dưới 15 từ), không chứa link web và không có các từ khóa lộ liễu như `win, free, cash`. Ví dụ: "*Did you receive my previous document? Please reply to confirm.*"
- **Nhận xét:** Với email quá ngắn và thiếu tín hiệu văn bản, mô hình NLP dựa vào tần suất từ khó phân biệt nếu không có thêm thông tin về người gửi (địa chỉ IP, tên miền, bản ghi SPF).

9. TỔNG KẾT & GIỚI HẠN CỦA BÀI LAB

- **Hoàn thành mục tiêu:** Bài lab đã thực hành đầy đủ chu trình học máy cơ bản, tự cài đặt các thuật toán From-Scratch đạt kết quả tốt trên tập dữ liệu thực nghiệm (Test Accuracy 96.52%, Recall 95.21%).
- **Hạn chế về dữ liệu:** Dữ liệu chỉ gồm email tiếng Anh từ một nguồn Kaggle cố định; chưa có các trường siêu dữ liệu riêng biệt như tiêu đề (Subject) hay thông tin người gửi (Sender/Recipient).
- **Hạn chế về điểm số xác suất:** Điểm số trả về của Linear SVM qua hàm sigmoid chưa được hiệu chuẩn (Uncalibrated). Do đó, điểm 0.8 chỉ mang tính xếp hạng tương đối so với ngưỡng 0.555 chứ không phải là xác suất toán học 80% spam.
- **Kiểm chứng mã nguồn:** Toàn bộ 57/57 unit tests trong `tests/` đều pass, bảo đảm mã nguồn không bị lỗi số học, không tràn số và giữ đúng thứ tự đặc trưng khi nạp lại mô hình.